

SELQIE Lite

Standard Operating Procedures

Ryan Kaczmarczyk

Created on 11/25/25

Last Modified on 2/18/26

Annotated Edition — 7/13/26

Expanded with implementation detail and a glossary drawn directly from the **SELQIE-LITE** ROS 2 workspace (source code and README) so operators can see what each command and subsystem actually does under the hood. Original procedure text is preserved verbatim; additions are marked in blue "From the codebase" callout boxes.

Power System

Battery

SELQIE Lite uses a MaxAmps 6S 4500 mAh Lithium-Ion Battery Pack. There are two batteries located in the labeled green case, #1 and #2 respectfully. They can be charged using the iCharger DX12 located in the FabLab. Do not leave the battery unattended while it is charging. The following settings are critical to the safety of charging the battery:

Critical Specifications

Property	Value
Battery type	Lithium-Ion
Battery capacity	4500 mAh
Charge type	Balance Charge
Charge rate	2.25 A (0.5C Max)
Nominal/storage voltage	21.6 V
Maximum voltage	25.2 V
Minimum operating voltage	18 V [Do Not Load the Battery While at or Below This Voltage*]
Minimum voltage	15 V

Additional Information is available on the Official MaxAmps Datasheet. This can be found online or in the SELQIE Lite Binder.

*Voltage checks should be performed every half hour while the robot is powered.

FROM THE CODEBASE — BATTERY TELEMETRY

Voltage checks do not have to be done purely by multimeter. The `battery` package's `tinybms_voltage_uart` node polls the pack's **TinyBMS** battery management system over UART (`/dev/ttyTHS1` @ 115200 baud, request command `0x14` , CRC-16/MODBUS-checked) and publishes the live pack voltage on the ROS 2 topic `/tinybms/ack_voltage` (`std_msgs/Float32`) at 2 Hz by default. The SELQIE Lite Terminal's `battery` command (see [Additional Terminal Commands](#)) prints this cached reading along with its age in seconds — a fast way to satisfy the half-hour voltage-check requirement without opening the case. Note that in the current `selqie_hw.launch.py` bring-up file this node is commented out by default and must be launched explicitly (`ros2 launch battery tinybms_voltage_uart.launch.py`) if live telemetry is needed.

Charging

Charging for SELQIE Lite will generally take place within an onboard charging configuration. In this arrangement, the lid will not need to be opened to charge it.

Onboard Charging

GAP IN SOURCE PROCEDURE

The original document ends this subsection at the heading with no further steps. The onboard charging port, cabling, and step-by-step charging sequence are mechanical/electrical hardware details that live in the SELQIE Lite Binder rather than in the software repository, so no additional detail could be sourced from the codebase for this step. Confirm the physical charging procedure against the binder before charging on-robot.

Operation

Powering on & off SELQIE Lite

Locate the power switch on the front of the lid. To power on, rotate the knob **clockwise** until it stops. To power off, rotate the knob **counterclockwise** 1 full rotation.

Connecting via Ethernet

To connect to SELQIE Lite's onboard computer, the ethernet cable should be connected to the laptop via the USB-C to ethernet adapter. Ensure that unnecessary stress is not put onto the tether. In some cases, this may require taping the non-SELQIE Lite side of the tether down to your working surface.

Running the SELQIE Lite Terminal

To connect to SELQIE Lite from the SELQIE laptop, type the following code into terminal:

```
$ ssh ciscor@10.42.0.2
```

When prompted for a password, enter the following:

```
$ jecAME109
```

At this point, you should now be inside SELQIE Lite's home directory.

FROM THE CODEBASE — ONBOARD COMPUTER

The host at `10.42.0.2` is an **NVIDIA Jetson Orin Nano** running Ubuntu 22.04 with **ROS 2 Humble**. The workspace on the robot is the ROS 2 *colcon* overlay `selqie_lite_ws` (this `SELQIE-LITE` repository lives in its `src/` directory); it must be built with `colcon build --symlink-install` and sourced (`source install/setup.bash`) before any `ros2` command will resolve.

Launching SELQIE Lite TERMINAL

To launch selqie terminal, first navigate to the tmux folder of the workspace.

```
$ cd selqie_lite_ws/src/tmux
```

Run the selqie.sh file using the command below.

```
$ ./selqie.sh
```

FROM THE CODEBASE — WHAT SELQIE.SH ACTUALLY DOES

`tmux/selqie.sh` starts a detached **tmux** session named `selqie`, splits it into a 2x2 grid, sources the workspace overlay in every pane, and launches one job per pane:

- **Pane 0 (top-left):** `ros2 launch selqie_bringup selqie_hw.launch.py`
- **Pane 1 (bottom-left):** a plain sourced shell
- **Pane 2 (top-right):** `ros2 run selqie_ui selqie_terminal`
- **Pane 3 (bottom-right):** `jtop` (jetson-stats)

It then loads `tmux/.tmux.conf`, which turns on mouse support for click-to-select panes/scrolling and binds `prefix + x` to send `SIGINT` to every pane before killing the session — the fast, clean way to stop all motors/gaits/recordings and tear down the session at once.

A second script, `tmux/selqie_glider.sh`, launches the same 2x2 layout but runs `ros2 run selqie_ui selqie_glider` in place of `selqie_terminal`. This starts an unattended **glider/mission controller** (`selqie_glider.py`) that watches the reed switch and drives the status LEDs through a red → flashing → green sequence to confirm the lid latch is closed before a mission begins. In the current codebase most of the autonomous mission body (standing, running the Beuhler gait, re-zeroing) is commented out pending further development — use `selqie.sh` for normal interactive operation.

About the SELQIE Lite Terminal

The test environment consists of four windows.

1. **Top left window:** status updates [where the nodes are running]
2. **Top right:** SELQIE Lite TERMINAL [Used for controlling SELQIE Lite]
3. **Bottom left:** standard Linux terminal.
4. **Bottom right:** Jetson GUI [Jetson Orin Nano Temperature, CPU Usage, etc.]

FROM THE CODEBASE — WHAT "THE NODES" IN THE TOP-LEFT PANE ARE

`selqie_hw.launch.py` brings up the hardware driver stack whose startup and runtime logs appear in the top-left pane:

Node / launch include	Package	Role
<code>servo_motor_node</code> ×4	<code>servo</code>	One CAN driver per leg motor (CubeMars AK-series actuator), exposes <code>/motorX/servo_cmd</code> , <code>/motorX/special_cmd</code> , <code>/motorX/motor_state</code> , <code>/motorX/error_code</code>
Leak sensor	<code>leak_sensor</code>	GPIO leak detector, publishes <code>/leak_detected</code>
LED driver	<code>ws2812b_ros</code>	Drives the WS2812B status LEDs from <code>/led_colors</code>
Depth sensor	<code>ms5837_bar_ros</code>	Bar30 pressure/depth sensor over I ² C, publishes <code>/bar30/depth</code> , <code>/bar30/depth_salt</code> , etc.
IMU	<code>bno08x_driver</code>	BNO08x 9-DOF IMU, publishes <code>/imu</code>

Two entries in that launch file are currently **commented out** and will not start automatically: the **latch node** (`latch` package — servo latch + reed switch over USB serial) and the **battery voltage node** (`battery`

package, described above). If a procedure needs the physical latch driven from the terminal or live pack-voltage telemetry, launch those packages by hand first, e.g. `ros2 launch latch latch.launch.py` and `ros2 launch battery tinybms_voltage_uart.launch.py`.

Zeroing the legs

Verify that all 4 legs are pointing down and can be moved with little resistance. In the SELQIE Lite Terminal, run the command

```
SELQIE> zero
```

Then, to enter stance:

```
SELQIE> stand
```

FROM THE CODEBASE — WHAT ZERO / STAND DO ON THE WIRE

`zero` first sends each targeted motor an `idle` command (servo mode 7, so the leg can be moved by hand without fighting the motor), then a `zero` special command. The motor driver (`servo_motor_node.py`) answers `zero` by sending a **set-origin CAN packet** that redefines the encoder's current physical position as the new relative zero — this is a destructive re-reference, so the legs must actually be pointed straight down first (as the procedure states) or the new "zero" will be wrong. `stand` then starts a 50 Hz position-control loop (`StandHold`) that continuously commands all four legs to position 0° (servo mode 4) until another gait or `idle` is issued.

Additional Terminal Commands

As documented in the original procedure:

- **status** – prints the motor statuses to the terminal window
- **idle** – allows the motors to move freely
- **battery** – prints the current voltage of the onboard battery
- **start_recording** – starts a ros2bag recording of available data
- **stop_recording** – stops a ros2bag recording of available data

FROM THE CODEBASE — EXACT BEHAVIOR, PLUS UNDOCUMENTED COMMANDS

Command	Usage	What it does
<code>status</code>	<code>status</code>	Prints each leg's cached <code>MotorState</code> : <code>position</code> (rad), <code>abs_position</code> (deg), <code>velocity</code> (rad/s), <code>torque</code> (Nm), <code>current</code> (A), <code>temperature</code> (°C).
<code>idle</code>	<code>idle</code> <code>[motor_id all]</code>	Sends servo mode 7 (duty = 0) — motor coasts freely, no holding torque.
<code>battery</code>	<code>battery</code>	Prints the last cached <code>/tinybms/pack_voltage</code> reading and its age in seconds.

<code>start_recording</code>	<code>start_recording</code> <code>[output_dir]</code>	Runs <code>ros2 bag record</code> on every motor cmd/state/error topic (all 4 legs) plus <code>/imu</code> , <code>/bar02/depth</code> , <code>/bar30/depth</code> , <code>/bar30/depth_salt</code> , <code>/latch_angle_cmd</code> , <code>/led_colors</code> , and <code>/tinybms/pack_voltage</code> . If no directory is given it auto-names one under <code>rosbags/selqie_<YYYYMMDD_HHMMSS></code> .
<code>stop_recording</code>	<code>stop_recording</code>	Sends <code>SIGINT</code> to the recording process and waits (up to 10 s, then force-terminates) so the bag closes cleanly.
<code>clear</code>	<code>clear</code> <code>[motor_id all]</code>	<i>(Not in original SOP.)</i> Stops any running gait/stand loop and holds the motor at idle — a soft "pause".
<code>errors</code>	<code>errors</code>	<i>(Not in original SOP.)</i> Prints the last fault string reported by each motor driver (see Error Codes appendix below).
<code>brake</code>	<code>brake</code> <code><current_a></code> <code>[motor_id all]</code>	<i>(Not in original SOP.)</i> Applies a fixed regenerative braking current (servo mode 2) to slow a leg without commanding a position.
<code>snap</code>	<code>snap</code> <code>[motor_id all]</code>	<i>(Not in original SOP.)</i> Rounds a leg's current position to the nearest multiple of 360° and holds it there — useful for recovering from encoder wind-up without a full re-zero.
<code>latch</code>	<code>latch</code> <code>open close</code>	<i>(Not in original SOP.)</i> See Latch Control below.
<code>exit</code>	<code>exit</code>	<i>(Not in original SOP.)</i> Stops any active recording, stops all gaits, and cleanly shuts down the ROS 2 node before leaving the terminal — the preferred way to leave the SELQIE Lite Terminal.

The Swimming Gait

The format to run the swimming gait is as follows:

```
SELQIE> swim <frequency_hz> [center_angle] [delta_angle]
```

The items in `< >` are required. The items in `[ ]` can be left blank and will assume default values. Angles should be entered in degrees.

To stop the swimming gait, run:

```
SELQIE> swim stop
```

Then run:

```
SELQIE> zero
```

Followed by:

```
SELQIE> stand
```

FROM THE CODEBASE — SWIM GAIT MATH & DEFAULTS

`swim` drives all four legs in unison with a simple sinusoid at 50 Hz: $\text{position}(t) = \text{center_angle} + \text{delta_angle} \times \sin(2\pi \times \text{frequency_hz} \times t)$. Default `center_angle` is 60° and default `delta_angle` is 75° if left blank; a default frequency of 1.5 Hz is used internally by the gait class, though the terminal requires you to supply one explicitly. `swim stop` halts the oscillator, restarts the motors, and immediately re-engages `stand` (position 0°) — so in practice the terminal's `swim stop` already re-stands the robot; the follow-up `zero / stand` pair in this procedure is a belt-and-suspenders re-reference step, not strictly required to stop the motion.

The Walking Gait (Beuhler)

The format to run the walking gait is as follows:

```
SELQIE> beuhler <frequency_hz> [slow_band_deg] [alpha]
```

- **slow_band_deg** — the width of the angle in which the leg moves slower
- **alpha** — the ratio of the faster region velocity to the slower region of velocity

The items in `< >` are required. The items in `[ ]` can be left blank and will assume default values. Angles should be entered in degrees.

To stop the walking gait, run:

```
SELQIE> beuhler stop
```

Then run:

```
SELQIE> zero
```

Followed by:

```
SELQIE> stand
```

FROM THE CODEBASE — BEUHLER CLOCK GAIT DETAIL

The name refers to a **Buehler clock**-style gait — a clock-like leg trajectory (named for legged-robotics researcher Martin Buehler) that alternates a slow "stance" phase with a fast "swing" phase around a single rotation. In this implementation the four legs are split into two diagonal pairs — group A (legs 1 & 4) and group B (legs 2 & 3) — offset by 180° so the pairs step out of phase. Each leg spends `slow_band_deg` (default 30°)

of its rotation in the slow band centered on 0° (stance) and the remaining 330° in the fast band (swing); `alpha` (default **1.5**) sets how much faster the swing phase moves than stance. `frequency_hz` is the overall cycle frequency (default **0.5 Hz** internally) and leg speed is capped at 20 rad/s. `beuhler_stop` snaps each leg to the nearest multiple of 360° and holds it there before the gait thread exits, which is why the procedure's follow-up `zero` / `stand` re-references the legs to a clean pose afterward.

Latch Control (new section — not in original procedure)

SELQIE Lite's pressure-housing lid is secured by a motorized latch. It can be driven directly from the SELQIE Lite Terminal:

```
SELQIE> latch open
SELQIE> latch close
```

FROM THE CODEBASE — LATCH HARDWARE PATH

`latch open` publishes `0°` and `latch close` publishes `180°` on the `/latch_angle_cmd` topic (`std_msgs/Float64`). The `latch` package's `latch_node` forwards that angle over USB serial (`/dev/ttyACM0` , 115200 baud) to a **Teensy microcontroller**, which drives a **Hitec D954SW** servo to the commanded angle. The same Teensy reports the mechanical **reed switch** state back up the serial link (`REED 0` / `REED 1`), which `latch_node` re-publishes as `/reed_switch` (`std_msgs/Bool`) so software can confirm the latch is physically seated closed, not just that the servo was commanded closed. A separate `reed_switch` package exists that reads the same style of magnetic switch directly from a Jetson GPIO pin (board pin 16) instead of through the Teensy, publishing `/reed_switch_closed` — use whichever is wired on a given build.

As noted above, the latch node is **commented out** of the default `selqie_hw.launch.py` bring-up, so `latch open` / `latch close` will have no effect until it is launched separately (`ros2 launch latch latch.launch.py`).

Safety Sensors (new section — not in original procedure)

LEAK DETECTION

The `leak_sensor` package polls a GPIO leak probe (board pin 33, active-high) at 10 Hz and publishes `/leak_detected` (`std_msgs/Bool`). `True` means moisture has been detected inside the housing. Because this node is part of the standard `selqie_hw.launch.py` bring-up, it starts automatically with `selqie.sh` ; watch the top-left status pane for a "LEAK" state-change log line during operation, and treat any transition to `LEAK` as an immediate abort condition — power off and open the housing.

Two more sensors add situational awareness while submerged, both started by the standard hardware bring-up:

Sensor	Package	Interface	Publishes
Bar30 pressure/depth (300 m rated)	<code>ms5837_bar_ros</code>	I ² C	<code>/bar30/pressure</code> , <code>/bar30/temperature</code> , <code>/bar30/depth</code> , <code>/bar30/depth_salt</code> , <code>/bar30/odom</code>

BNO08x 9-DOF IMU	<code>bn08x-ros2-driver</code>	I ² C/UART	<code>/imu</code> (orientation/attitude for state estimation)
------------------	--------------------------------	-----------------------	---

Shutting Down the Orin

In a terminal window, run the following code to shut down the Orin.

```
$ sudo shutdown -h now
```

Enter the password

```
$ jecAME109
```

RECOMMENDED ORDER

Type `exit` in the SELQIE Lite Terminal pane first (or press `prefix + x` in tmux to `SIGINT` every pane) so any active `ros2 bag` recording is closed cleanly and all gaits are stopped before the Orin's OS shuts down. Killing the session or power switch without exiting first can leave a bag file unflushed/corrupted and skips the motor idle step.

Appendix A — Motor Command Modes

Reference for the underlying `/motorX/servo_cmd` (`std_msgs/Float64MultiArray` : `[mode, v0, v1, v2]`) values that terminal commands ultimately publish. Not required for day-to-day operation, but useful for debugging with `ros2 topic pub` directly.

Mode	Name	Fields used	Effect
0	duty	<code>v0</code> = duty (-1..1)	Open-loop duty cycle drive
1	current	<code>v0</code> = amps	Closed-loop current (torque) control
2	current_brake	<code>v0</code> = brake amps	Regenerative braking current (<code>brake</code> command)
3	rpm	<code>v0</code> = ERPM	Closed-loop electrical-RPM speed control
4	position	<code>v0</code> = degrees	Closed-loop position control (used by <code>stand</code> , <code>swim</code> , <code>beuhler</code> , <code>snap</code>)
5	set_origin	<code>v0</code> = 0/1/2	Re-zero encoder origin (temp / permanent / restore default) — underlies the <code>zero</code> special command
6	pos_spd	<code>v0</code> =deg, <code>v1</code> =ERPM, <code>v2</code> =accel	Position control with explicit speed/acceleration limits
7	idle	—	Duty forced to 0 — free-spin (used by <code>idle</code> , and transiently by <code>zero</code> / <code>clear</code>)

Separately, `/motorX/special_cmd` (`std_msgs/String`) accepts the text commands `start` , `exit` , `zero` , and `clear` , which power the CAN transceiver on/off or trigger the zero/idle-hold behaviors described above.

Appendix B — Motor Error Codes

Reported per leg on `/motorX/error_code` and surfaced by the `errors` terminal command.

Code	Meaning
0	No error
1	Over temperature fault
2	Over current fault
3	Over voltage fault
4	Under voltage fault
5	Encoder fault
6	Phase current unbalance fault (hardware may be damaged)

Glossary

Terms are defined as they are actually used in the SELQIE-LITE ROS 2 workspace (`README.md` and source), not generically, so definitions match this robot's implementation.

abs_position

Field on the `MotorState` message reporting a leg's absolute encoder position in degrees, distinct from `position` (radians, relative/wrapped).

AK-series actuator

CubeMars brushless "AK" line of geared servo actuators used for the four leg joints (e.g. `AK10-9`, `AK40-10`, `AK70-10`, `AK80-64` appear in the driver's torque-constant table). Each has an integrated CAN-based driver. A full reference manual ships in the repository root as "*AK Series Driver and Control Manual v1.0.15.X.pdf*."

Bar30 / Bar02

Blue Robotics MS5837-based pressure/depth sensors (300 m and 10 m rated, respectively) read over I²C by the `ms5837_bar_ros` package; used to report depth, pressure, temperature, and a derived odometry message.

Beuhler clock (Buehler clock)

The walking gait invoked with `beuhler`. A clock-inspired leg trajectory that spends most of a rotation in a fast "swing" arc and a short, slower "stance" arc near a fixed reference angle, with diagonal leg pairs offset 180° from each other. See [The Walking Gait \(Beuhler\)](#) above.

BNO08x

SparkFun 9-DOF inertial measurement unit (IMU) integrated via the `bn08x-ros2-driver` submodule; publishes orientation data on `/imu`, used for state estimation.

CAN / can0 / SocketCAN

Controller Area Network — the differential serial bus (interface name `can0` on the Jetson, via Linux's SocketCAN framework) that carries motor commands and telemetry between the Jetson and each leg's CubeMars driver.

colcon

The ROS 2 build tool used to compile every package in the workspace (`colcon build --symlink-install`).

ERPM (electrical RPM)

The speed unit used by the VESC-style servo command protocol (mode 3 "rpm" and the speed field of mode 6 "pos_spd"). ERPM is mechanical RPM multiplied by the motor's pole-pair count, so it differs from the physical shaft speed.

Idle (motor mode 7)

Servo command mode that forces duty to 0, letting a leg spin/move freely with no holding torque. Triggered by the terminal's `idle` command and used transiently inside `zero` / `clear`.

Jetson Orin Nano

The NVIDIA single-board computer that serves as SELQIE Lite's onboard computer, reachable at `10.42.0.2` over the ethernet tether. Its temperature, CPU, and GPU load are shown live by `jtop` in the terminal's bottom-right pane.

jtop (jetson-stats)

Command-line/TUI system monitor for Jetson boards; this is the "Jetson GUI" referenced in the original procedure's terminal-layout description.

Latch

The motorized mechanism securing SELQIE Lite's pressure housing lid, driven by a Hitec D954SW servo via a Teensy microcontroller. Commanded from the terminal with `latch open` (0°) / `latch close` (180°); see [Latch Control](#).

Leak sensor

GPIO-based moisture probe inside the housing; publishes `/leak_detected`. Any `True` reading is a safety abort condition. See [Safety Sensors](#).

LED status strip (WS2812B / NeoPixel)

Addressable RGB LEDs driven over SPI by the `ws2812b_ros` package (topic `/led_colors`, packed 24-bit `0x00RRGGBB` values per LED). Used by the terminal and the glider mission controller to show status at a glance (e.g. red = powered/waiting, orange/flashing = waiting on the latch reed switch, green = latch confirmed closed).

MIT-mode vs. Servo-mode

Two different low-level command protocols for CubeMars motor drivers referenced in this codebase. MIT-mode (position/velocity/feed-forward torque packed into one CAN frame) is described in the top-level README's `quad_legs` package documentation; the hardware stack actually wired into `selqie_hw.launch.py` and the SELQIE Lite Terminal today uses the simpler VESC-style **Servo-mode** protocol implemented in the `servo` package.

MotorState

Custom ROS 2 message type (`motor_interfaces/msg/MotorState`) reporting one leg motor's `name`, `position` (rad), `abs_position` (deg), `velocity` (rad/s), `torque` (Nm), `current` (A), and `temperature` (°C). This is what the terminal's `status` command prints.

Reed switch

A magnetically-actuated switch used to sense whether the latch is physically seated closed (as opposed to merely commanded closed). Reported either via the Teensy (`/reed_switch`, through the `latch` package) or directly from a Jetson GPIO pin (`/reed_switch_closed`, via the standalone `reed_switch` package).

ros2bag / ros2 bag record

ROS 2's built-in data-recording tool. The terminal's `start_recording` / `stop_recording` commands wrap it to capture every motor, sensor, and command topic to a timestamped folder for later playback or analysis.

Servo-mode / servo_cmd

The generic per-leg command topic `/motorX/servo_cmd` (`std_msgs/Float64MultiArray`, `[mode, v0, v1, v2]`) that all terminal motion commands (`stand`, `swim`, `beuhler`, `idle`, `brake`, `snap`) ultimately publish to. See Appendix A for the full mode table.

special_cmd

The per-leg text command topic `/motorX/special_cmd` (`std_msgs/String`) accepting `start`, `exit`, `zero`, and `clear` — power-on/off and re-zero operations that don't fit the numeric `servo_cmd` format.

Stand (StandHold)

A background 50 Hz control loop that continuously commands all four legs to a fixed position (0° by default) — how the terminal's `stand` command keeps the robot in a stable stance between gaits.

Swim gait

The oscillatory swimming gait invoked with `swim`: all four legs move in unison through a sinusoidal position trajectory around a center angle. See [The Swimming Gait](#) above.

Tether

The ethernet cable connecting the operator's laptop to SELQIE Lite's onboard computer, via a USB-C-to-ethernet adapter on the laptop end.

TinyBMS

The battery management system monitoring the 6S Li-ion pack. Polled over UART by the `battery` package to report live pack voltage on `/tinybms/ack_voltage`.

tmux

Terminal multiplexer used to present the four-pane operator layout (`tmux/selqie.sh`, `tmux/.tmux.conf`) in a single SSH session, with mouse support and a one-key "stop everything" binding (`prefix + x`).

Zero (re-origin)

The act of redefining a leg's current physical position as its new relative zero, performed by sending a CAN "set origin" packet. Triggered by the terminal's `zero` command; must be done with the leg physically pointed straight down, as the procedure specifies, or all subsequent position commands will be offset.

ZED camera

Stereolabs stereo depth/tracking camera integrated via the `zed-ros2-wrapper` and `zed-ros2-examples` submodules, connected over USB 3.x, for depth and vision streams.

This annotated edition preserves every procedural step and warning from the original SELQIE Lite Standard Operating Procedures document verbatim; all "From the codebase" callouts, the Appendices, the Safety Sensors and Latch Control sections, and the Glossary were added by cross-referencing the `SELQIE-LITE` repository's `README.md` and source (packages: `servo`, `selqie_ui`, `motor_interfaces`, `battery`, `latch`, `reed_switch`, `leak_sensor`, `sensing`, `ws2812b_ros`, `selqie_bringup`, `tmux`).